

Walter: A Hexapod Platform for Open-Loop Gait Optimization and MuJoCo PPO Locomotion Experiments

Howard Wang
Engineering Department
Swarthmore College
Swarthmore, PA USA

Matthew Shiley
Engineering Department
Swarthmore College
Swarthmore, PA USA

Abstract

Walter (Walking Alternating Tripod for Evolutionary Research) is a twelve-degree-of-freedom hexapod with 3D-printed structure, hobby servos, custom Hexapod V2 PCBs, a Teensy-class MCU, eighteen-channel servo driver, and BMI270 IMU. We built a reproducible workflow: CAD and assembly, a hand-tuned discrete tripod baseline, a reduced-order surrogate with L-BFGS-B optimization whose output exports without manual transcription to open-loop sinusoidal firmware, and a MuJoCo PPO training and evaluation toolchain (Stable-Baselines3) with archived checkpoints and short evaluation reports. We tested the surrogate optimizer (forward motion about 0.23 m/s mean over a steady window, ≈ 1.9 m travel in 8 s on the lumped model) and ran representative PPO evaluations in simulation (task-specific displacements; rewards not comparable across configs). Adaptive IMU/reflex firmware and sim-to-real transfer of learned policies remain future work. Tracked purchased cost is US\$792.67 (three electronics platforms plus three PCB/stencil orders).

Index Terms

Hexapod robot, gait optimization, firmware integration, reinforcement learning, MuJoCo, PPO, legged locomotion.

I. INTRODUCTION

Reliable legged locomotion coordinates periodic limb motion with feedback under limited sensing and computation. Insects suggest a *layered* picture—rhythmic coordination, fast feedback, slower adjustment—that we use only as an organizing design heuristic [1], [2], not as an experimentally validated controller on the physical robot.

What this report focuses on. The implemented contributions are the hardware platform, an open-loop optimization-to-firmware pipeline with deterministic parameter parity, and a reusable MuJoCo PPO training and evaluation stack. Layered adaptive firmware (IMU fusion, reflexes, online parameter adaptation) remains future work on the MCU; simulation observation choices are informed by that roadmap [3], [4], but we do not claim those layers on hardware.

Lifelong and resource-aware learning for field robots [5] motivates why we care about compact open-loop laws and sim-trained policies. Our engineering response is concrete: a low-dimensional alternating-tripod rhythm (inspired by common hexapedal templates) is optimized in a fast surrogate, frozen in firmware, and complemented by PPO in MuJoCo for state-conditioned torques [6]–[8]. Domain shift and latency mean simulation results are not automatic predictors of field behavior; closing the loop on the robot is the next hard step.

The physical system is *Walter* (Sec. VI-D–VI-E). Sec. II gives optimization and PPO background. Sec. III–IV and Table I summarize delivered and partially delivered work; Sec. V states proposed vision and distance-centric extensions. Code and this document are at <https://github.com/HowardWHSrun/WALTER>.

II. MATHEMATICAL BACKGROUND: OPTIMIZATION AND REINFORCEMENT LEARNING

The project uses two complementary ideas. Offline gait optimization asks which interpretable, periodic joint law moves the body forward under acceptable oscillation and loading *proxies* when feedback is not yet in the loop. Reinforcement learning in simulation asks whether a *state-conditioned* policy can improve behavior under task rewards when the agent senses proprioception, base motion, and (in some configs) IMU channels. The first searches a small vector ϕ on a fast surrogate; the second searches neural weights in MuJoCo.

A. Reduced-order gait optimization (Track 1)

a) *Problem and modeling choice.*: Walter has twelve actuated joints; the space of conceivable gait waveforms is enormous, and hand tuning is slow. We restrict attention to a *compact* alternating-tripod sinusoidal law: resting abduction and flexion per leg, per-leg phase shifts, a common stride frequency, and fixed swing-lift structure. That choice mirrors a biological design pattern—encode locomotion with a low-dimensional rhythm before adding sensory correction—and matches what the open-loop firmware can execute, so optimized parameters transfer directly to C. The remaining design question is purely quantitative: find ϕ in a feasible box that yields good forward motion in a *reduced-order* planar model. Full MuJoCo rollouts could in principle score each ϕ , but the surrogate integrates an eight-state body + leg coupling quickly enough for many L-BFGS-B iterations and random restarts, which supports systematic search rather than one-off hand edits. The scalar cost $\mathcal{J}(\phi)$ below turns informal goals (move forward, stay comparatively smooth, avoid extreme loading proxies, stay near defaults) into a single objective for the optimizer.

1) *Decision variables and bounds*: With that parameterization, the optimizer acts on $\phi \in \mathbb{R}^{25}$ matching `hexapod_no_imu_optimization`. Indices are concatenated as

$$\phi = (\theta_{\text{abd,rest}}^\top, \theta_{\text{flx,rest}}^\top, \delta_{\text{abd}}^\top, \delta_{\text{flx}}^\top, f)^\top, \quad (1)$$

where each of $\theta_{\text{abd,rest}}, \theta_{\text{flx,rest}}, \delta_{\text{abd}}, \delta_{\text{flx}} \in \mathbb{R}^6$ are in *degrees* in the code (converted to radians internally), indexed by leg order FL, FR, ML, MR, RL, RR, and $f \in \mathbb{R}_+$ is the common gait frequency in hertz. Box constraints are implemented as `scipy.optimize.Bounds`: for example $f \in [0.4, 4.0]$ Hz; resting abduction per joint in $[-25^\circ, 25^\circ]$; resting flexion in $[10^\circ, 65^\circ]$; phase shifts in $[-120^\circ, 120^\circ]$. These define a feasible orthotope $\phi_{\min} \leq \phi \leq \phi_{\max}$.

2) *Open-loop joint commands (sinusoidal law)*: Let $\omega = 2\pi f$. For leg $\ell \in \{1, \dots, 6\}$ with fixed tripod offset $\psi_\ell \in \{0, \pi\}$ (alternating tripod), time $t \geq 0$, and fixed amplitude tables $\bar{a}_{\text{abd},\ell}, \bar{a}_{\text{flx},\ell}$ (defaults in code, not decision variables), the commanded abduction and flexion angles are

$$\phi_{\text{abd},\ell}(t) = \theta_{\text{abd,rest},\ell} + \bar{a}_{\text{abd},\ell} \sin(\omega t + \psi_\ell + \delta_{\text{abd},\ell}), \quad (2)$$

$$\phi_{\text{flx},\ell}(t) = \theta_{\text{flx,rest},\ell} + \bar{a}_{\text{flx},\ell} \sin(\omega t + \psi_\ell + \delta_{\text{flx},\ell}) - \Delta_{\text{lift},\ell}(t), \quad (3)$$

where $\Delta_{\text{lift},\ell}$ implements swing-phase foot clearance (subtracting flexion when the abduction sine is positive), identical in structure to the firmware `LIFT_SWING_DEG_PEAK` term. Velocities $\dot{\phi}_{\text{abd},\ell}, \dot{\phi}_{\text{flx},\ell}$ are obtained by differentiation and feed the planar surrogate dynamics.

3) *Surrogate planar dynamics and simulation*: The reduced model integrates an eight-dimensional state

$$\mathbf{x}(t) = (x, \dot{x}, z, \dot{z}, \phi_{\text{roll}}, \dot{\phi}_{\text{roll}}, \phi_{\text{pitch}}, \dot{\phi}_{\text{pitch}})^\top, \quad (4)$$

with $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \phi)$ assembled from lumped masses, soft ground contact, leg thrust and support forces derived from $\phi_{\text{abd},\ell}, \phi_{\text{flx},\ell}$ and contact weights, and linear damping on body motion and attitude rates (`body_ode` in the script). For fixed ϕ , the trajectory is obtained by `solve_ivp` (Runge–Kutta) over $t \in [0, T]$ with T the simulation horizon (`sim_duration_s`), sampled at N time points (`sample_count`).

4) *Scalar objective (explicit weighted sum)*: Let a subscript “ss” denote statistics evaluated on a *steady-state* suffix of the horizon (the code uses the last 60% of samples for oscillation metrics and mean speed). Write $\bar{v}_x$ for mean forward velocity $\mathbb{E}_{\text{ss}}[\dot{x}]$, and RMS values $\text{RMS}_{\text{ss}}[\phi_{\text{roll}}], \text{RMS}_{\text{ss}}[\phi_{\text{pitch}}], \text{RMS}_{\text{ss}}[z]$. Let $q_{\text{servo}} \geq 0$ be a slack on peak torque proxy exceeding a threshold (0.85 of stall in the implementation), and σ_{contact} the time mean of the per-timestep standard deviation of contact weights across legs. With nonnegative weights w , `matching OptimizationConfig`, the implemented cost is

$$\begin{aligned} \mathcal{J}(\phi) = & -w_{\text{fwd}} \bar{v}_x + w_{\text{back}} [\max(0, -\bar{v}_x)]^2 \\ & + w_{\text{rp}} \left(\text{RMS}_{\text{ss}}[\phi_{\text{roll}}]^2 + \text{RMS}_{\text{ss}}[\phi_{\text{pitch}}]^2 \right) + w_z \text{RMS}_{\text{ss}}[z]^2 \\ & + w_{\text{servo}} q_{\text{servo}}^2 + w_{\text{contact}} \sigma_{\text{contact}}^2 + w_{\text{reg}} \|\phi - \phi_0\|_{\text{scaled}}^2, \end{aligned} \quad (5)$$

where ϕ_0 is the default parameter vector and $\|\cdot\|_{\text{scaled}}$ averages coordinatewise squared normalized deviation from the box midpoint (Tikhonov-style regularization keeping ϕ in a “reasonable” neighborhood). Optional terms penalize shortfall below a target mean speed if `target_forward_speed_mps` > 0. Infeasible simulations return a large penalty (10^6). The weighted sum encodes engineering tradeoffs: prioritize forward progress while penalizing backward drift, attitude and vertical oscillation, actuator overload proxies, uneven leg loading, and deviation from nominal ϕ_0 , which can admit energetic gaits with visible body motion in the surrogate.

Weights are heuristic. The coefficients w in (5) reflect priorities chosen for this milestone, not a uniquely justified physical performance measure. Different weights would generally yield different gaits; sensitivity analysis is future work.

5) *Optimization algorithm and restarts*: The problem solved in software is

$$\min_{\phi} \mathcal{J}(\phi) \quad \text{subject to} \quad \phi_{\min} \leq \phi \leq \phi_{\max}. \quad (6)$$

L-BFGS-B (`scipy.optimize.minimize`, method L-BFGS-B) approximates curvature of $\mathcal{J}$ with a limited-memory inverse Hessian and projects each iterate onto the box. Each evaluation of $\mathcal{J}$ requires one numerical integration of the surrogate. The outer loop runs R random restarts (`restarts`): one start at ϕ_0 and $R - 1$ uniform samples in the box; the best ϕ^* by lowest $\mathcal{J}$ is retained. Limits `maxiter`, `maxfun`, and `ftol` cap work per restart.

Core contribution (deterministic parity). A main outcome of Track I is *algebraic parity* between surrogate optimization and flashed open-loop firmware: ϕ^* serializes to JSON, a host script emits a C header, and the sketch evaluates the same `leg_commands()` map—eliminating manual transcription from analysis to deployment. Bench degree-to-pulse gains and clamps remain separate calibration layers.

B. Policy optimization in MuJoCo (PPO with GAE)

a) *Problem and modeling choice.*: Open-loop ϕ^* from the surrogate is a single time-varying schedule: it cannot react to unexpected contact or tilt. For behaviors that depend on *current* state, we optimize a policy $\pi_{\theta}(a | s)$ in MuJoCo. The objective is standard: maximize expected discounted return $\mathbb{E}[\sum_t \gamma^t r_t]$ under transition dynamics from the simulator and a user-defined reward that encodes forward progress, stability penalties, control cost, and task structure (paths, terrain, etc.). The thinking is to let the agent *adapt* torques to observations—including IMU-related channels when configured—so performance is not locked to one precomputed waveform. That comes at the cost of sample complexity, sim-to-real gap, and opaque policy weights compared to the explicit ϕ vector; the math below is the PPO/GAE machinery Stable-Baselines3 uses to optimize θ stably.

Legged RL is cast as an MDP with MuJoCo supplying transitions; we use actor-critic PPO as implemented in Stable-Baselines3 [8]. Generalized advantage estimation (GAE) [6] builds advantages from TD residuals $\delta_t = r_t + \gamma V_{\psi}(s_{t+1}) - V_{\psi}(s_t)$:

$$\hat{A}_t = \sum_{\ell=0}^{\infty} (\gamma \lambda)^{\ell} \delta_{t+\ell}, \quad (7)$$

with λ and discount γ set in YAML. **PPO** [6] stabilizes updates via a clipped ratio between the new and behavior policy. Define the **probability ratio**

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \quad (8)$$

For continuous actions, π_{θ} is typically Gaussian; r_t is the ratio of densities. The **clipped surrogate objective** for a batch of timesteps is

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right], \quad (9)$$

with clip range $\epsilon \in (0, 1)$ (often 0.2). Large policy steps that inflate r_t when $\hat{A}_t > 0$ (or shrink r_t when $\hat{A}_t < 0$) are clipped so the effective objective stops increasing, reducing catastrophic updates common in vanilla policy gradients.

Let $\hat{R}_t$ denote a target return for the critic (e.g., n -step return or GAE-backed value target). With coefficients $c_v, c_e > 0$, the **composite loss** maximized in practice is the negative of

$$\mathcal{L}_{\text{total}}(\theta, \psi) = -\mathcal{L}^{\text{CLIP}}(\theta) + c_v \mathbb{E}_t [(V_{\psi}(s_t) - \hat{R}_t)^2] - c_e \mathbb{E}_t [H(\pi_{\theta}(\cdot | s_t))], \quad (10)$$

where H is differential entropy for Gaussian policies. Minimizing $\mathcal{L}_{\text{total}}$ corresponds to ascending $\mathcal{L}^{\text{CLIP}}$ while fitting the critic and discouraging collapse of exploratory variance. Updates use minibatches over several epochs on each rollout buffer; hyperparameters (learning rate, rollout length `n_steps`, batch size, γ , λ , ϵ , c_v , c_e) are set in YAML under `hexapod_training_new/configs/`.

III. TRACK I: SURROGATE OPTIMIZATION AND FIRMWARE EXPORT

This track **implements** Sec. II-A: the ϕ parameterization, sinusoidal `leg_commands()`, the eight-state surrogate, cost (5), box-constrained L-BFGS-B with restarts, and deterministic JSON-to-header-to-sketch parity (Sec. II-A). The workflow isolates a tunable open-loop rhythm before closed-loop MCU layers are required.

What we shipped. Optimizer script, summary JSON, JSON-to-header sync, and open-loop Arduino sketch with exported gains. Quantitative surrogate outputs appear in Tables VII–VIII; the MuJoCo asset used for RL is documented in Fig. 4.

IV. TRACK II: PPO LOCOMOTION IN MUJoCo

The policy $\pi_\theta(a|s)$ conditions on **proprioception** (joint positions and velocities), simulator **base kinematics**, and—in IMU-focused YAML configs—**accelerometer and gyroscope** channels (or complementary-filter-style stacks) so roll, pitch, and body rates align with how *Walter* is sensed on hardware. **Tripod structure** enters through phase features, structured policy heads (e.g., sCPG variants), or reward shaping, depending on the experiment. Training uses Eqs. (7)–(9) on MuJoCo rollouts.

Scope. The team **built, trained, and evaluated** policies in simulation, archived checkpoints, and wrote short evaluation summaries for representative runs (Table IX). Deploying a trained policy as the primary closed-loop driver on the physical robot—with the same observation latency and actuation path as firmware—was **out of scope** for verification in this report.

TABLE I

SCOPE AND STATUS OF MAJOR TECHNICAL THREADS FOR *Walter*. ^AINFRASTRUCTURE AND REPRESENTATIVE SIM BEHAVIORS, NOT A CLAIM OF STRONG CROSS-TASK LOCOMOTION (TABLE IX).

Thread	Status	Technical core	Notes
Open-loop surrogate → firmware	Delivered	$\mathcal{J}(\phi)$ (5); L-BFGS-B; JSON→.h→ sketch	Deterministic parameter parity
MuJoCo PPO toolchain	Delivered ^a	PPO (9); GAE (7); configs, checkpoints, evals	Reproducible workflow
Vision / distance-centric RL	Proposed	Features f_t , rewards on forward displacement, expanded s_t	Sec. V; not in repo

V. FUTURE WORK: VISION INTEGRATION AND DISTANCE-CENTRIC REWARDS

For production-style deployment, observations may eventually fuse proprioception and IMU with visual odometry or pose from an overhead or onboard camera so rewards emphasize distance traveled with less simulator-privileged state. Let $p_t \in \mathbb{R}^2$ be horizontal pose from visual integration or fiducials, and consider forward displacement $x_t - x_0$. A natural reward increment is

$$r_t = \alpha(x_{t+1} - x_t) - \beta \|\omega_t\|^2 - \rho \text{penalty}_{\text{fall}} - \kappa \text{TV}(a)_t, \quad (11)$$

with IMU rates ω_t penalizing harsh attitude changes, and optional terms for staying upright. The observation becomes $s_t = [s_t^{\text{IMU/proprio}}, f_t]$ where f_t carries visual features or estimated pose; PPO would reuse (9) with expanded input dimension. This stack requires sim-to-real work on vision and latency, reward design from estimated motion, and validation on hardware [5]—**future work** beyond the delivered repository snapshot.

VI. TECHNICAL DISCUSSION

A. Biological and control context

Decentralized feedback atop rhythmic coordination is standard in insect locomotion [1], [2]. IMU and posture literature [3], [4], [9] informs *simulation* observation design; disturbance-triggered transitions [10] motivate reflex ideas in documentation, not validated firmware here.

B. Open-loop sinusoidal gait (implemented)

For the no-IMU milestone, each leg follows resting abduction and flexion angles plus sinusoidal variation at common frequency f , with tripod group B offset by π from group A. This is the same law optimized in Sec. II-A and delivered in Track I (Sec. III). Amplitudes follow default tables shared by Python and `optimized_gait_params.h`.

C. MuJoCo and reinforcement learning (simulation)

High-fidelity training uses Gymnasium + MuJoCo [7] with PPO [6] as implemented in Stable-Baselines3 [8]; the mathematical objective is Eq. (9) with advantages (7). Track II (Sec. IV) specializes observation and reward; Sec. V outlines a vision-augmented extension. YAML configs set γ , λ , learning rate, network widths, PD gains, and terrain. Fig. 4 documents the simulation asset (Sec. IX-B); Fig. 5 shows a policy-controlled frame from a released evaluation video.

D. Mechanical design: CAD and 3D-printed parts

Walter’s chassis and legs are built from **parts designed in CAD** and produced by **additive manufacturing** (3D printing): a central body, six leg assemblies, and two-degree-of-freedom linkages per leg (abduction and flexion) that mate to servo splines. Additive manufacturing kept iteration cost low and allowed internal cable routing and tray mounts that would be tedious in stock materials alone, at the expense of part stiffness and tolerance stack-up that we managed with pilot holes, shoulder screws, and shims where the linkage meets horns. The layout prioritizes twelve independent actuators, wiring clearance, and a tray for the battery and PCB. Figure 1 shows an overall render; Figure 2 shows subassemblies. Fasteners and servo hardware complete the stack; filament mass and printed-part dollar cost are not included in Tables III–V.

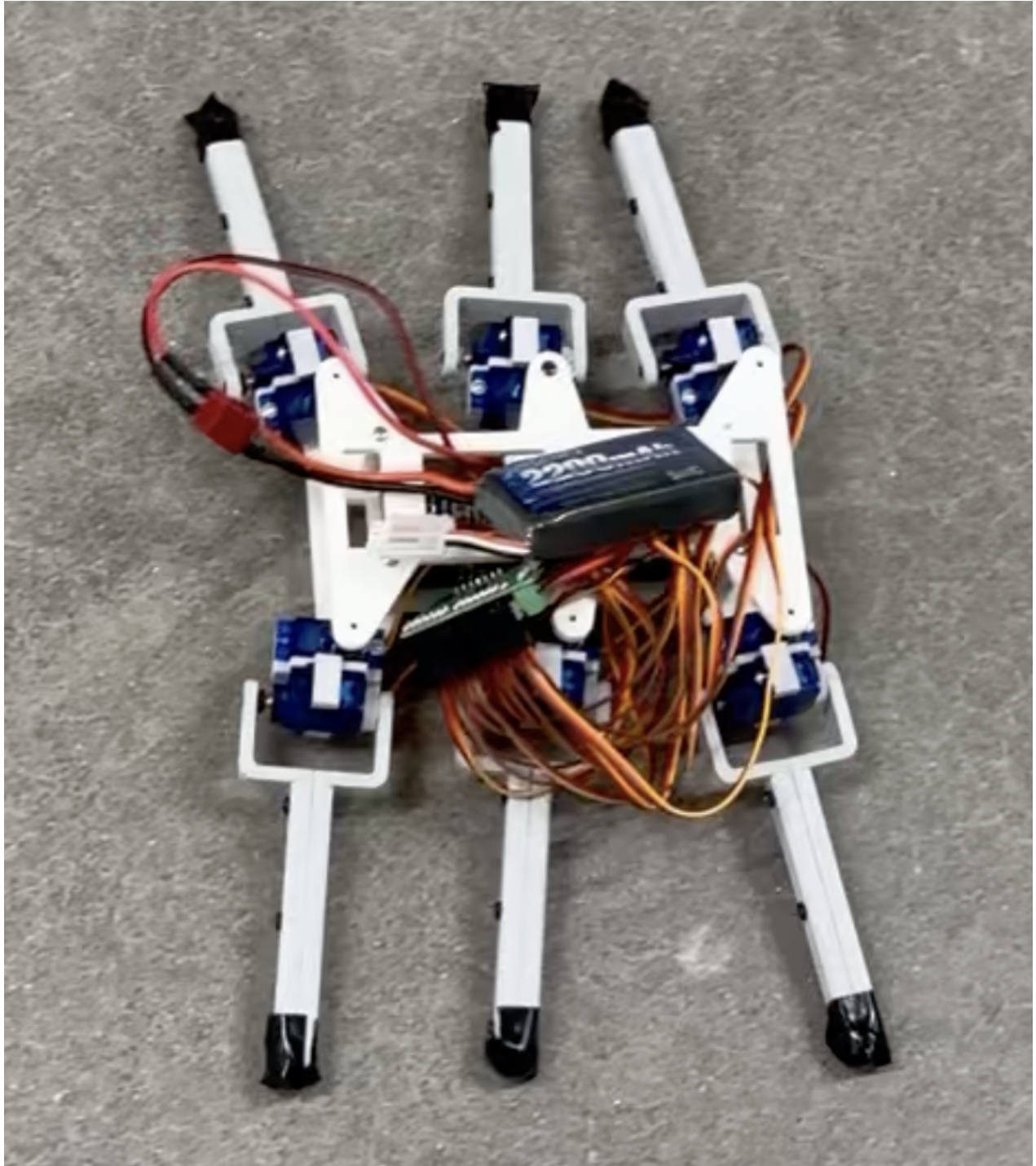


Fig. 1. Walter: full-assembly render of the twelve-DOF hexapod (CAD; Walking Alternating Tripod for Evolutionary Research).

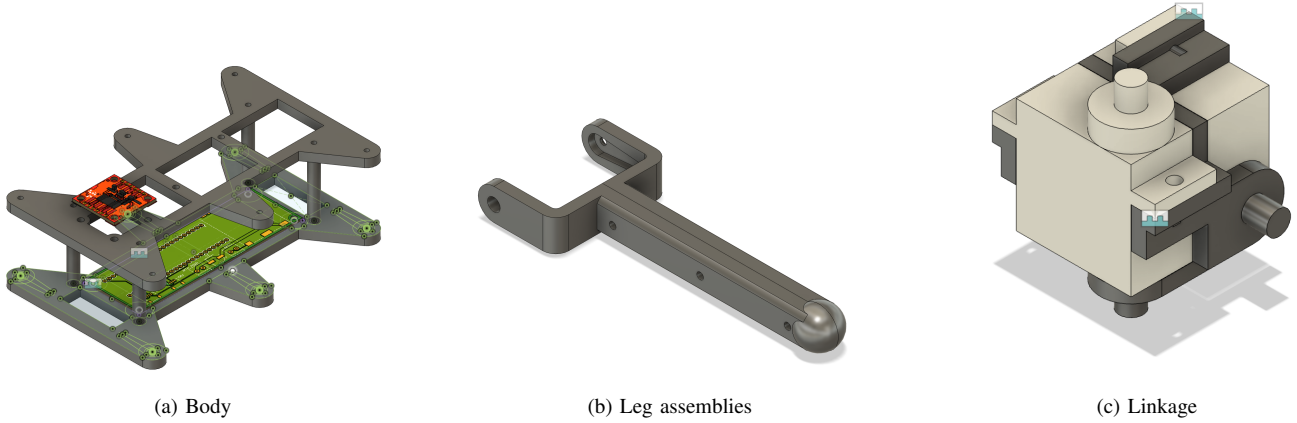


Fig. 2. CAD-designed 3D-printed subassemblies for Walter: chassis, legs, and per-leg linkage.

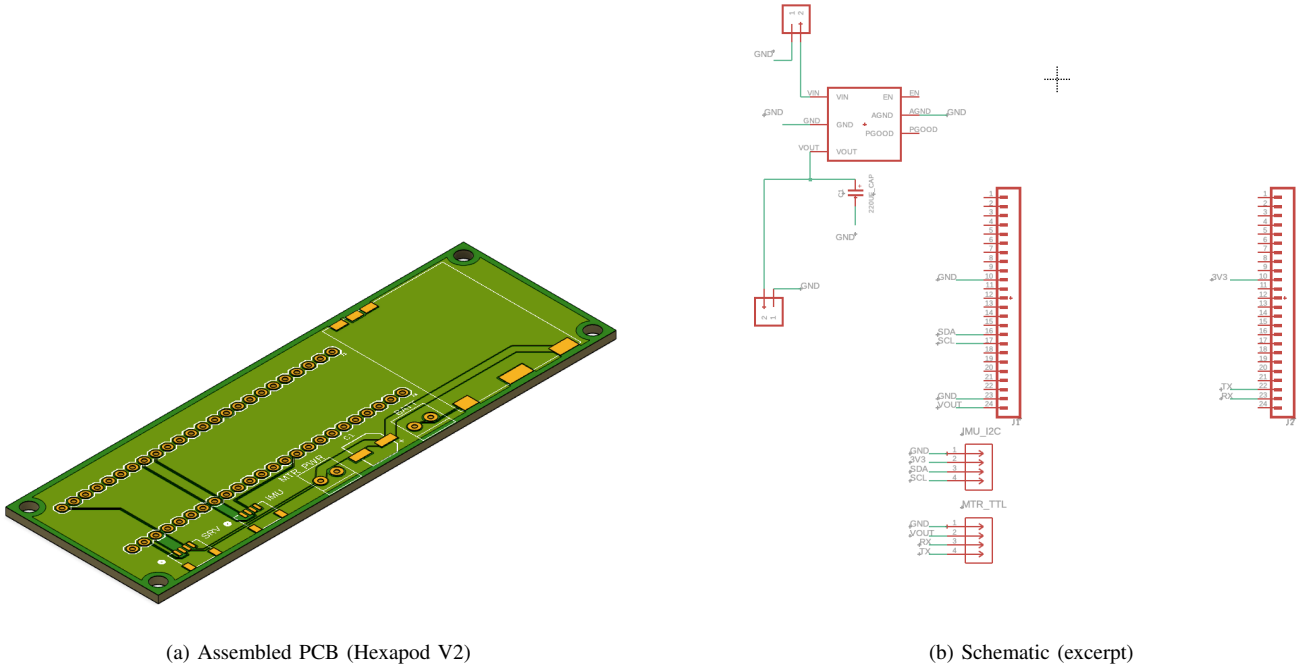


Fig. 3. Custom electronics for Walter: Hexapod V2 board and schematic.

E. Electronics, Hexapod V2 PCB, and safety

The bill of materials matches *purchased* components: Teensy-class MCU (SparkFun 16771), Pololu 1354 eighteen-channel driver, buck converter, BMI270 via Qwiic cabling, dual-cell LiPo and charger, terminals, and servos. Custom *Hexapod V2* PCBs consolidate power and signal routing so the wiring harness stays short and repeatable; revision v5 (dated 2026-04-03 on the latest order) reflects iterative fixes to trace width, connector footprints, and servo header grouping learned from earlier bring-up. Figure 3 shows a board photo and schematic excerpt. `docs/FIRMWARE.md` records UART to the servo controller, channel ordering FL/FR/ML/MR/RL/RR with abduction then flexion per leg, and pulse mapping $u_k = \text{clamp}(c_k + d_k q_k)$ (typical bounds 600–2400 μs). Open-loop firmware applies degree-to-command gains and clamps before streaming compact-protocol targets; those bench gains are *not* outputs of the surrogate optimizer and were the main manual tuning step between optimized angles and reliable motion.

VII. REQUIREMENTS AND CONSTRAINTS

A. Source of requirements

Engineering requirements are traced to phase exit criteria in docs/PHASES.md (Phases 1–4: IMU telemetry, reflexes, cautious mode and recovery, online adaptation), supplemented by safety and interface constraints in docs/FIRMWARE.md and the open-loop scope in firmware/hexapod_openloop_sine/README.md. PHASES.md serves as the team’s consolidated requirements-and-constraints specification for firmware behavior.

B. Summary: met vs. not met

Satisfied in this project: discrete tripod baseline firmware; open-loop optimization with JSON export and header sync; calibration sketches; CAD-documented mechanical design (Figs. 1–2); a MuJoCo PPO workflow with configs, training scripts, checkpoints, and evaluation writeups. **Not satisfied against the internal phased firmware spec:** Phases 1–4 adaptive behaviors (telemetry-only through online adaptation) are largely **not implemented** on the MCU in the snapshot summarized below.

TABLE II
REQUIREMENT TRACEABILITY (INTERNAL PHASES.md VS. DELIVERED WORK).

ID	Requirement	Status	Evidence / notes
R1	Phase 1: IMU read, complementary filter, USB telemetry ~50 Hz; gait unchanged	Not met (in repo)	PHASES.md; folder may be missing
R2	Phase 2: postural reflexes, clamps, serial tuning, EEPROM	Not met (in repo)	ARCHITECTURE.md pseudocode
R3	Phase 3: stability score, cautious mode, push recovery, fall freeze	Not met (in repo)	Documented only
R4	Phase 4: five-parameter adaptation, cost J , SPSSA/bandit, EEPROM	Not met (in repo)	Documented only
R5	Hardware: MCU + eighteen-channel driver + servos + documented map + PCBs + Walter mechanical CAD/3D parts	Met	BOM, Figs. 1–3, FIRMWARE.md
R6	Safety: pulse clamps; parameter slew (adaptive layer)	Partial	Clamps in v2/openloop; slew in spec
R7	Open-loop: optimizer/firmware parity; JSON export	Met	Optimizer, sync_gait_params_from_json.py
R8	Simulation: MuJoCo env, PPO training, documented evals	Met	hexapod_training_new/, e90_repo/released_checkpoints/

C. Adaptive firmware roadmap (design documentation only)

Internal docs/ARCHITECTURE.md and PHASES.md describe a two-timescale stack (fast IMU/reflex/servo tick; slower adaptation). That roadmap informed simulation observation modes but is **not** part of the validated MCU implementation in this report (Table II). A nominal complementary-filter-style attitude blend appears in Appendix A.

D. Project cost

Tables III–IV summarize **purchased** electronics (three identical platforms) and **fabrication** costs for three PCB/stencil orders. Table V states the **grand total** for those categories only; filament, fasteners, and shop labor are outside the accounting. Table III lists **eight** purchased hobby servos per platform; the additional actuators required for twelve independently driven joints were obtained outside that tracked line item (e.g., existing lab inventory) and are therefore **not** included in the dollar subtotal.

VIII. METHODS

A. Mechanical assembly (Walter)

Printed body and leg components are fastened per the CAD design; each leg receives two servos driving abduction and flexion through the linkage. The custom PCB mounts to the chassis and connects to the Pololu driver and Teensy; power and ground respect the buck converter and screw terminals from the BOM. Bench alignment uses the published servo zeroing utility and side-only motion tests before open- or closed-loop gait trials.

B. Baseline discrete gait

Legacy walking firmware implements a deterministic alternating-tripod **state machine**: each tripod group holds a small set of key joint poses, and the scheduler steps through stance/swing segments with smoothed interpolation (moveSmoothTo-style motion). It was the first runnable gait on hardware: quick to debug, easy to intuition-tune for “walking at all,” and independent of the surrogate. Limitations are typical of open-loop keyframe gaits: time-consuming to improve, no systematic forward-speed objective, and no export path from the optimizer. The optimized sinusoidal law (Track I) replaces that schedule with a single analytic rhythm tuned on the surrogate; PPO (Track II) replaces both with a state-conditioned torque policy in simulation (Table VI).

TABLE III
ELECTRONICS BILL OF MATERIALS, **ONE PLATFORM** (USD). THE TEAM PURCHASED THREE IDENTICAL SETS.

Part	Description	Vendor	Qty	Unit	Line
SparkFun 16771	Microcontroller	DigiKey	1	37.20	37.20
Pololu 1354	18-channel servo driver	DigiKey	1	46.95	46.95
DFRobot DFR1202	Step-down buck converter	DFRobot	1	12.90	12.90
SparkFun BMI270	6-DoF IMU	DigiKey	1	18.50	18.50
SparkFun 11855	2×7.4 V LiPo + charger	Amazon	1	27.99	27.99
SparkFun 14417	Female Qwiic connector	DigiKey	2	0.60	1.20
SparkFun 17259	100 mm Qwiic cable	DigiKey	1	1.95	1.95
SparkFun 17261	Qwiic cable to female header	DigiKey	1	1.95	1.95
Amphenol	1×2 screw-down terminal	DigiKey	2	1.28	2.56
YO0201500000G					
DFRobot SER0039	Servo motor (tracked purchase)	DigiKey	8	5.90	47.20
See Sec. VII-D: eight servos are costed here; four more per robot for twelve DOF were sourced outside this line.					
Subtotal	one platform				198.40
Electronics total	three platforms (×3)				595.20

TABLE IV
PCB AND STENCIL ORDERS, *Hexapod V2* (USD ORDER TOTALS INCLUDING MERCHANDISE, SHIPPING, DUTIES/TAXES, AND SALES TAX WHERE CHARGED BY THE VENDOR).

Order ID	Description / notes	Status	Total
Y4-11911448A	PCB prototype (5 pcs) + SMT stencil; v5_2026-04-03	Awaiting payment	74.24
W2026030804205410	PCB + stencil shipment	Shipped	68.26
W2026021709554129	PCB + stencil (earlier order)	Shipped	54.97
PCB subtotal			197.47

C. Reduced-order simulation optimization

Gait parameters are tuned on a planar surrogate of the hexapod. The toolchain integrates the surrogate with SciPy’s `solve_ivp`, evaluates a weighted objective on a steady-state time window (forward speed versus roll, vertical motion, actuator load, and contact-balance penalties), and runs box-constrained L-BFGS-B minimization with random restarts. Each run writes a JSON summary of optimized frequencies, rest angles, phase shifts, and scalar kinematic metrics; optional plots support debugging.

D. Parameter sync and open-loop firmware

The microcontroller sinusoidal gait runs at a fixed control period (10 ms in the released configuration), with startup frequency ramping, degree-to-command mapping, output clamping, and serial commands to the servo driver. A small host-side step merges the optimizer JSON into a generated C header so the flashed sketch uses the same numbers as the surrogate optimum without manual transcription.

E. Bench testing and calibration

Zeroing and side-only checks precede full gait tests. Documentation in the open-loop package explains sign and gain adjustments when feet drag or the body shuffles backward.

F. MuJoCo PPO training and evaluation

The simulation stack trains PPO from YAML experiment definitions: each run records TensorBoard summaries, policy checkpoints, and logs in a dedicated run directory, and a matching evaluation driver reloads the same configuration to produce

TABLE V
GRAND TOTAL PROJECT COST (USD) FOR REPORTED PURCHASES AND FABRICATION.

Category	Amount
Electronics (three platforms)	595.20
PCB fabrication and stencils (three orders)	197.47
Grand total	792.67

short rollout statistics and human-readable summaries. Released archives include representative checkpoints and evaluation folders for the results tables in this report.

IX. RESULTS

TABLE VI
QUALITATIVE COMPARISON OF CONTROLLER STRANDS IN THIS PROJECT (NOT A UNIFORM QUANTITATIVE BENCHMARK).

	Discrete baseline	Optimized open-loop	PPO (simulation)
Feedback	None (time schedule)	None (time schedule)	State-conditioned
Tuning	Hand keyframes	Surrogate + L-BFGS-B	Policy optimization
Primary evidence	Bench gait	Surrogate metrics + firmware parity	Archived evals / video

A. Reduced-order optimization (surrogate model)

Table VII summarizes the committed surrogate optimizer summary file. These numbers describe the *planar surrogate* only; they are not a guarantee of on-robot speed or stability.

TABLE VII
OPTIMIZER-REPORTED METRICS (COMMITTED SUMMARY JSON). DOMINANT FREQUENCY: ARGMAX OF THE POWER SPECTRUM (SCI-PY PERIODOGRAM) APPLIED TO VERTICAL VELOCITY $\dot{z}$ ON THE STEADY-STATE SUFFIX OF THE ROLLOUT (LAST 60% OF SAMPLES, MATCHING THE OPTIMIZER).

Quantity	Value
Gait frequency f	1.80 Hz
Mean forward speed (steady window)	0.234 m/s
Total displacement (rollout)	1.86 m
Roll RMS	0.440 rad
Pitch RMS	~ 0 (near zero in file)
Vertical position RMS	0.086 m
Peak servo usage ratio (proxy)	0.152
Contact balance std. dev.	0.388
Dominant frequency ($\dot{z}$ periodogram)	3.54 Hz
Objective cost $\mathcal{J}(\phi^*)$	-228.38

Reading Table VII. The optimizer achieves forward motion in the reduced model (≈ 0.23 m/s mean, ≈ 1.9 m over 8 s), which validates the **optimization-to-firmware pipeline** under the stated surrogate and weights. Roll RMS near 0.44 rad is large: with the current w_{rp} , the objective still **under-penalizes** attitude oscillation relative to forward speed, so the solution is useful as a transfer proof, not as evidence of deployment-quality attitude behavior. Pitch RMS near zero reflects planar symmetry of the lumped model. Coarse contact and torque proxies mean hardware will differ.

How far, how fast, and how the legs move. Stride cadence is 1.8 Hz (≈ 0.56 s per cycle). Joint targets are sinusoidal about the rest poses in Table VIII with fixed abduction/flexion amplitudes in the parameterization: outer legs $\pm 14^\circ/\pm 18^\circ$, middle legs $\pm 10^\circ/\pm 20^\circ$ (FL-RR order); tripods are 180° out of phase; swing clearance subtracts up to 12° flexion at peak abduction phase. The $\dot{z}$ dominant line in Table VII reports spectral content of vertical velocity on the steady window; it is a diagnostic of the lumped contact model, not a second gait clock.

Table VIII lists ϕ^* merged into the exported firmware header. **Diagnostic (bound saturation):** all six flexion phase offsets δ_{flx} sit at the lower bound (-90°). That pattern usually signals that the optimum lies outside the box, that the phase parameterization is too restrictive, or that the weighted objective pushes coarsely on phasing; it is a cue for revisiting bounds, penalties, or model fidelity rather than for minor local retuning only. Rest angles and frequency remain plausible; bench tuning may relieve the bound.

TABLE VIII
OPTIMIZED DECISION VARIABLES ϕ^* FROM L-BFGS-B (SAME JSON AS TABLE VII). ANGLES IN DEGREES; LEG ORDER FL, FR, ML, MR, RL, RR.

Quantity	Value
Gait frequency f	1.800 Hz
Abduction rest $\theta_{\text{abd},\text{rest}}$	(10, 10, 0, 0, -10, -10)
Flexion rest $\theta_{\text{flx},\text{rest}}$	(32.00, 32.00, 34.00, 34.00, 32.00, 32.00)
Abduction phase δ_{abd}	≈ 0 (magnitude $< 10^{-6}$ deg)
Flexion phase δ_{flx}	(-90, -90, -90, -90, -90, -90)
Swing lift amplitude	12.0 deg (peak flexion reduction during swing)

B. MuJoCo simulation (high-fidelity model)

Track II training uses Gymnasium with the MuJoCo asset `hexapod_training_new/assets/hexapod.xml`: checkerboard ground, twelve torque-controlled joints, and RK4 integration at 0.002 s (see configs for frame skipping). Figure 4 is an **asset documentation** frame only: offscreen render after gravity settle with **zero** control (not a trained policy). It records mesh, materials, and contact setup used for rollouts.

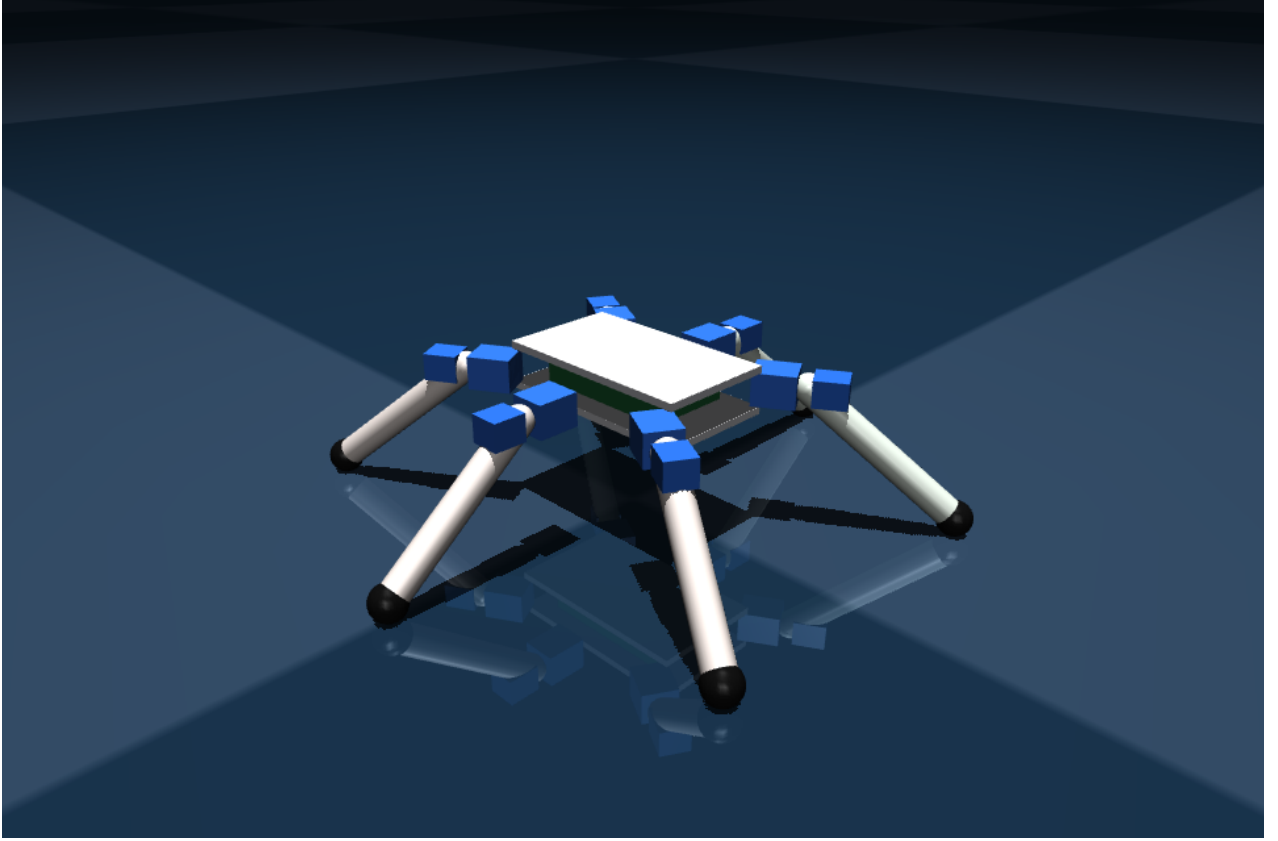


Fig. 4. MuJoCo asset for Walter (passive settle under zero torque; documents geometry and contact for training, not policy performance).

C. MuJoCo PPO evaluations (released checkpoints)

The PPO thread is best read as a **completed training and evaluation infrastructure**: reproducible configs, checkpoints, REPORT.md summaries, and short videos for representative policies. Table IX quotes means from released evaluation folders under `e90_repo/released_checkpoints/evaluations/`. Raw rewards are **not** comparable across configs (different shaping and scaling); forward displacement and path length are coarse readouts within each task family.

TABLE IX
REPRESENTATIVE PPO EVALUATION MEANS (300 ENV STEPS PER EPISODE UNLESS NOTED; SEE EACH REPORT.MD).

Checkpoint	Task / policy	Fwd. $+x$	2D path	Notes
mlp_clean_run005_ best	Circle path; MLP 256×256 PPO; training run mlp_run_005 (301k steps).	0.18 m	0.27 m	Mean reward ≈ 6800 .
terrain_low_shake_ run001_best	Flat, rough, and step terrain; MLP PPO.	0.33 m (3 ep. mean)	—	Flat/rough ≈ 0.47 m forward; steps harder.

Reading Table IX. These rows document **representative learned behaviors** under their respective rewards, not a unified locomotion leaderboard. The circle-path MLP run shows modest forward motion and path length consistent with path tracking; the terrain run shows shorter mean forward progress when steps dominate. Cross-task claims remain limited; sim-to-real transfer is future work.



Fig. 5. Representative policy-controlled frame from released evaluation video (mlp_clean_run005_best, episode 1, circle-path task; 300-step protocol).

The velocity-window structured-policy checkpoint archived as `pd_velocity_window_run018_best` shows small net displacement in its released short evaluation—useful as an archival datapoint, not a headline locomotion result (see that evaluation folder’s summary).

X. DISCUSSION

Delivered artifacts. The team produced a **reusable hexapod platform** (CAD, assembly, custom PCBs, tracked BOM), a **reproducible open-loop pipeline** with deterministic surrogate-to-firmware parity, and a **MuJoCo PPO toolchain** (configs, checkpoints, evaluation summaries, and video for representative policies).

Lessons from the surrogate. Large roll RMS under the chosen weights shows the reduced model still trades attitude disturbance for speed: the result is strongest as proof of the optimizer and JSON-to-header workflow, not as a certificate of high-quality hardware gait. Saturated flexion phases (Table VIII) flag objective or bound design, not just a local optimum.

Hardware bottlenecks. Surrogate, MuJoCo, and hardware disagree on contact, friction, and servo dynamics; bench pulse mapping remains the main manual layer between optimized angles and motion; open-loop schedules do not reject disturbances.

Simulation scope. MuJoCo policies demonstrate *in-sim* training and evaluation under stated rewards; they are not automatic predictors of field performance. The highest-leverage next experiment is a short hardware rollout of one checkpoint with matched observation timing, before adding vision-only objectives.

Adaptive firmware. Phased IMU/reflex/adaptation goals in internal docs remain **future work** on the MCU for this snapshot (Table II).

Advice for future teams. Lock a baseline gait and logging format before layering sensors; budget bench time after every simulation milestone; track PCB revisions and BOM multiples from week one.

XI. CONCLUSION

This project **established an end-to-end engineering workflow**: mechanical design and assembly, embedded open-loop execution, surrogate optimization with deterministic export to firmware, and a reusable MuJoCo PPO training and evaluation stack with archived artifacts. The main technical risk has shifted from “can the pieces be integrated?” to “how to close feedback

on hardware under latency, power, and sensing limits?” Layered adaptive control inspired the roadmap but remains future work, documented in Table II. Code and this report are maintained at <https://github.com/HowardWHSrun/WALTER>.

ACKNOWLEDGMENT

We thank our advisors Allan Moser, Francisco Valero-Cuevas, and Will Johnson for guidance on the E90 Project. We thank Jennifer Parisien, Engineering 90 coordinator, and Edmond Jaoudi, technician, for coordination, fabrication, and bench support. We thank Swarthmore Engineering and shop or lab staff who supported fabrication and testing. No funding sources beyond the Engineering department are declared for this accounting.

REFERENCES

- [1] Y. O. Aydin *et al.*, “Mechanosensory control of locomotion in animals and robots: Moving forward,” *Biomimetics*, vol. 8, no. 4, p. 211, 2023, pMC10445419.
- [2] G. Chen *et al.*, “Adaptive locomotion control of a hexapod robot via bio-inspired learning,” *Frontiers in Neurorobotics*, vol. 15, p. 627157, 2021.
- [3] R. G. Valenti, I. Dryanovski, and J. Xiao, “Keeping a good attitude: A quaternion-based orientation filter for IMUs and MARGs,” *Sensors*, vol. 15, no. 8, pp. 19 302–19 330, 2015.
- [4] L. Bai *et al.*, “Development and implementation of a new approach for posture control of a hexapod robot to walk in irregular terrains,” *Robotica*, vol. 37, no. 12, pp. 2225–2238, 2019.
- [5] D. Kudithipudi, M. Aguilar-Simon, J. Babb, M. Bazhenov, D. Blackiston, J. Bongard, A. P. Brna, S. C. Raja, N. Cheney, J. Clune *et al.*, “Biological underpinnings for lifelong learning machines,” *Nature Machine Intelligence*, vol. 4, no. 3, pp. 196–210, 2022.
- [6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [7] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.
- [8] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, pp. 1–8, 2021.
- [9] R. Zhang *et al.*, “Attitude estimation algorithm of portable mobile robot based on complementary filter,” *Micromachines*, vol. 12, no. 11, p. 1373, 2021.
- [10] A. Johnson, K. D. Gregg, and D. E. Koditschek, “Disturbance detection, identification, and recovery by gait transition in legged robots,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2010, pp. 3627–3634.

APPENDIX A

NOMINAL ATTITUDE BLEND (DESIGN REFERENCE)

Documentation references a complementary-style update on roll (pitch analogous), e.g.

$$\alpha \in (0, 1), \quad \theta_{\text{roll}} \leftarrow \alpha(\theta_{\text{roll}} + \omega_x \Delta t) + (1 - \alpha) \theta_{\text{roll, accel}}. \quad (12)$$

This is **not** validated MCU firmware in the reported snapshot; it is retained as a design reference only.

APPENDIX B

PUBLIC REPOSITORY

The public project URL is <https://github.com/HowardWHSrun/WALTER>. Appendix paths below refer to the local Engineering 90 workspace layout at the time of writing; the GitHub repository is intended to subsume or mirror these components for reproducibility.

APPENDIX C

REPOSITORY MAP (SELECTED LOCAL PATHS)

- **Adaptive firmware and docs:** RL Temp/adaptive_onddevice_hexapod/README.md, docs/DESIGN.md, docs/ARCHITECTURE.md, docs/PHASES.md, docs/FIRMWARE.md
- **Firmware sketches:** firmware/hexapod_walking_v2/, firmware/hexapod_openloop_sine/, firmware/set_zero/, side-only tests
- **Reduced-order optimization:** hexapod_brain_roadmap copy/ode_optimization_no_imu/
- **MuJoCo PPO training:** RL Temp/hexapod_training_new/ (train.py, configs/, runs/)
- **Released checkpoints and evals:** RL Temp/e90_repo/released_checkpoints/
- **Presentation notes:** Presentation/E90_MidSemester_SpeakerScript.tex (and PDF)
- **This folder (figures):** walter_assembly.png, Body.png, Legs.png, Linkage.png, pcb_board.png, pcb_schematic.png, mujoco_hexapod_sim.png, ppo_mlp_run005_episode1_frame.png (plus originals with longer filenames)